Lab 2: Creating Normalized Tables with Constraints and Practicing DDL Commands

Learning Objectives:

- ✓ Identify data redundancy and anomalies in unnormalized tables.
- ✓ Apply normalization rules up to Third Normal Form (3NF).
- ✓ Design a normalized relational schema for a small database.
- ✓ Use DDL commands to create and modify database structures.
- ✓ Implement keys and constraints to enforce data integrity.

Learning Outcomes:

1. **Apply normalization rules (1NF, 2NF, 3NF)** to transform an unnormalized relation into a set of normalized tables.
2. **Design relational tables** that separate entities (e.g., Customers, Orders, Products) and minimize redundancy.
3. **Define and enforce table constraints**: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT.
4. **Use DDL commands** (CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE) to build and modify schema safely.
5. **Explain the impact of constraints and referential integrity** on data correctness and DDL operations (e.g., how foreign keys affect DROP/TRUNCATE).

Introduction:

Well-designed databases store data in a way that is efficient, consistent, and easy to maintain. Poor table design causes data redundancy and anomalies (insert, update, delete). This lab focuses on **normalization** — the process of organizing data into related tables — and practical use of **Data Definition Language (DDL)** to define schema and constraints that preserve data integrity.

You will model a small retail database, apply normalization rules, and practice DDL commands to create and refine the schema and constraints.

---

1. What is Normalization?

Normalization is a stepwise process that eliminates redundancy and ensures logical data dependencies. The common normal forms used in practice are:

- **First Normal Form (1NF)**
  - Ensure each column contains atomic (indivisible) values.
  - Eliminate repeating groups; each field holds a single value.
- **Second Normal Form (2NF)**
  - Achieve 1NF first.
  - Remove partial dependencies: every non-key attribute must depend on the whole (composite) primary key, not part of it.
  - Typically involves splitting tables that store attributes of sub-entities.

- **Third Normal Form (3NF)**
    - Achieve 2NF first.
    - Remove transitive dependencies: non-key attributes should not depend on other non-key attributes.
    - Ensures attributes relate only to the primary key.

Why to normalize?

- Eliminates update, insertion, and deletion anomalies.
- Reduces storage overhead from duplicate data.
- Makes maintenance (e.g., address change) consistent and simple.

Practical example (conceptual):

An unnormalized Sales table that repeats customer details for each order should be split into Customers, Orders, Products, and OrderItems (or Orders_Product) to avoid repetition and enable the database to represent relationships cleanly.

---

2. DDL Commands Overview:

DDL commands change the database schema. They are structural and usually auto-committed in many DBMSs.

- **CREATE TABLE** — define a new table with columns, data types, and constraints.
- **ALTER TABLE** — change an existing table: add/drop columns, add/drop constraints, modify column types.
- **DROP TABLE** — remove an entire table and its data.
- **TRUNCATE TABLE** — remove all rows while keeping the table structure (faster than DELETE; may be restricted by referential integrity).

---

3. Constraints (Rules that enforce data integrity)

Constraints are declarative rules applied at schema-level to ensure the data meets business logic.

- **PRIMARY KEY** — unique identifier for each row. Cannot be NULL and must be unique.
- **FOREIGN KEY** — enforces referential integrity; values must exist in the referenced primary key column. Optionally supports ON DELETE / ON UPDATE actions (CASCADE, SET NULL, RESTRICT).
- **NOT NULL** — requires a value in every row.
- **UNIQUE** — ensures no duplicate values in a column (or group of columns).
- **CHECK** — enforces a boolean condition on column values (e.g., Price > 0).
- **DEFAULT** — supplies a default when the insert omits the column.

Design considerations

- Use CHECK for simple business rules (positive quantities, valid status codes).
- Use UNIQUE for business identifiers (email, SKU).
- Foreign keys guard against orphaned child records; pick appropriate delete/update rules.

4. Examples:

Create Customers table (example)

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY KEY,
  FirstName VARCHAR(50) NOT NULL,
  LastName VARCHAR(50),
  Phone VARCHAR(20) UNIQUE,
  City VARCHAR(50),
  Country VARCHAR(50) DEFAULT 'Nepal'
);
```

Create Orders table with foreign key and CHECK example

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustID INT,
  Quantity INT,
  TotalAmount DECIMAL(10,2),
  OrderDate DATE,
  CONSTRAINT chk_total_positive CHECK (TotalAmount > 0),
  FOREIGN KEY (CustID) REFERENCES Customer(CustomerID)
);
```

Altering a table (adding a UNIQUE email column)

```
ALTER TABLE Customer
ADD Email VARCHAR (100) UNIQUE;
```

These examples show typical usage of DDL and constraints. Use the DBMS-specific syntax (MySQL, PostgreSQL, SQL Server) where small differences exist (e.g., ALTER COLUMN syntax).

---

5. Referential Integrity and DDL Interactions

- Attempting to drop a parent table referenced by a foreign key will fail unless the foreign key is dropped first or ON DELETE CASCADE was defined and you explicitly remove the parent rows.
- TRUNCATE may be prohibited on tables referenced by foreign keys (DBMS-dependent).
- Always check INFORMATION_SCHEMA or system catalogs to inspect constraints before altering schema.

Query example to view constraints (MySQL style):

```
SELECT tc.CONSTRAINT_NAME, tc.CONSTRAINT_TYPE, tc.TABLE_NAME, kcu.COLUMN_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
LEFT JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE kcu
```

```
    ON tc.CONSTRAINT_NAME = kcu.CONSTRAINT_NAME
WHERE tc.TABLE_NAME = 'orders' AND tc.TABLE_SCHEMA = DATABASE();
```

Lab Tasks: 2

1. **Create Customer Table**
   - Columns: CustomerID (Primary Key), FirstName (NOT NULL), LastName, Phone (UNIQUE), City, Country (DEFAULT 'Nepal').
2. **Create Orders Table**
   - Columns: OrderID (Primary Key), Quantity, Order_Status, TotalAmount (CHECK: > 0), OrderDate, CustID (Foreign Key → Customer.CustomerID).
3. **Create Products Table**
   - Columns: ProductID, Name, Description, Price.
4. **Create Orders_Product (OrderItems) Table**
   - Columns: OPID (Primary Key), OrdID, ProdID, plus quantity/price if desired for line-item detail.
5. **Alter Customer Table**
   - Add Email column and enforce UNIQUE constraint on it.
6. **Add Primary Key to Products**
   - Add or ensure ProductID is defined as the table's primary key.
7. **Enforce NOT NULL on Product Name**
   - Modify Products.Name so it **cannot be NULL**.
8. **Enforce Quantity CHECK**
   - Alter Orders.Quantity to ensure quantity > 0 (CHECK constraint).
9. **Enforce Price CHECK**
   - Alter Products.Price so it must be greater than zero.
10. **Add Foreign Key OrdID → Orders**
    - Add foreign key constraint on Orders_Product.OrdID referencing Orders.OrderID.
11. **Add Foreign Key ProdID → Products**
    - Add foreign key constraint on Orders_Product.ProdID referencing Products.ProductID.
12. **Add DOB to Customer**
    - Alter Customer to add DOB (Date of Birth) column.
13. **Attempt Duplicate Column Addition**
    - Attempt to add DOB again to observe the error raised by the DBMS and explain why it occurs.
14. **Drop DOB Column**
    - Remove DOB from Customer.
15. **Attempt to Drop Customer Table**
    - Attempt to drop the Customer table and note behaviors related to existing foreign key references (i.e., whether the DBMS blocks the operation).